COMP 3059 – Capstone Project I
Superstore Shift Management Website

Software Requirements Analysis and Design Assignment

This assignment is an overview to gather the software needs with requirements analysis and help to proceed with the design.

The requirements analysis helps to break down functional and non-functional requirements to a basic design view to provide a clear system development process framework. It involves various entities, including business, stakeholders and technology requirements.

The design is the activity following requirements specification and before programming. Software design usually involves problem solving and planning a software solution.

To work on this assignment, you could use the references and a sample template given below. The sample template can be customized to suit the nature of your project.

Reference Reading/Example to work on this Sprint document:
www.cse.msu.edu/~chengb/RE-491/Papers/SRSExample-webapp.doc

Source for this template:
www.tricity.wsu.edu/~mckinnon/cpts322/cpts322-srs-v1.doc

1.0 Introduction

The Introduction section provides an overview of the system using software requirements analysis and design for the scope of the system.

1.1 Purpose

This document describes the high level software requirements for the system. It describes the what, not how, of the capabilities of the system for the intended audiences.

The purpose of this document is to present a detailed description of the Shift Management System. It will explain the purpose and features of the system, the user and system interfaces, what the system will do, the constraints under which it must operate, and how the system will react to external events (e.g., shift changes, availability updates, time-off requests, and notifications). This document is intended for both the stakeholders and the developers of the system including employees, managers, and admin

1.2 Scope

This explains what the proposed system will and will not do. Describe relevant benefits, objectives and goals. The description of scope should be consistent with the Project Plan.

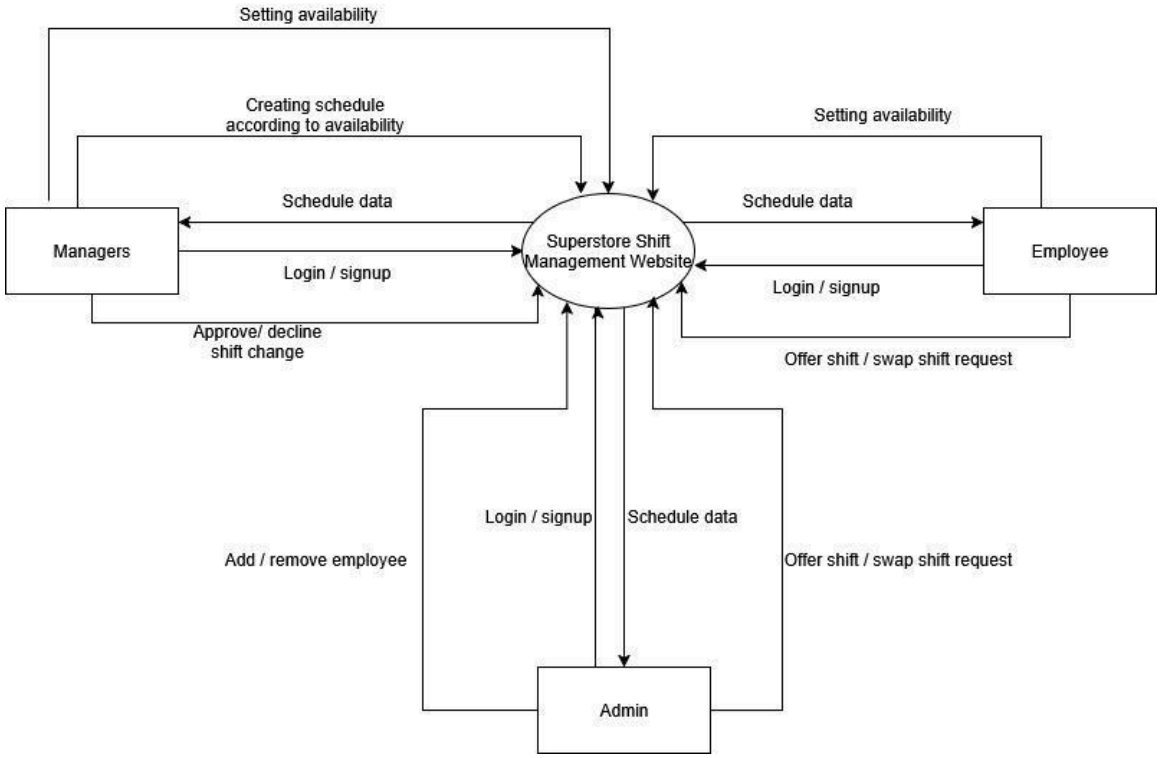
The scope of the Superstore Shift Management project is to deliver a functional web application with role-based authentication (for employees, managers, and administrators) within a two-semester timeframe, limited by a constraint of having no dedicated budget. The primary in-scope features focus on streamlining shift coordination by providing an employee dashboard to view schedules and submit availability, implementing a Shift Swap/Offer System, and enabling managers to create, publish, and approve all schedule and request changes via a dedicated interface.

2.0 System Overview

In the next chapter, **3.Functional Requirements** The System Overview section introduces the system context and design.

This overall description, presents a high level view of the **Shift Management System** it summarizes goals, primary users(employees, managers,admins), core workflows (scheduling, availability, swaps/offer shifts and approvals), and key constraints setting the context for the technical specification that follows.

The next chapter **4.Non-Functional Requirements** is aimed at developers. It defines the system in technical terms: detailed functional requirements and use cases, business rules, none-functional requirements and external integrations.



2.1 Project Perspective

The Project Perspective describes the context and origin of the system by defining whether the system is:

- a follow-on member of a system family
- a replacement for existing systems
- a new self-contained system.

The Shift Management System is a new self-contained system designed to streamline and simplify employee and shift coordinations. It is a web based application developed to address current inefficiencies in the manual scheduling process

2.2 System Context

The System Context describes the resulting software within the business case, including strategic issues in which the system is involved or which it specifically addresses.

- Setting small-mid size shift-base workplaces like cafes or retails, 10-20 staff,1-3 location
- Goal to cut scheduling time/errors ensure fair coverage and improve staff experience

- In-Scope: Web app + Api for employees, roles, locations, shifts, schedules and requests eg. offer/swap
- Out of scope: notification and time clock

- Success signals: 50% faster schedule creation, 2% less conflicts, 80% if request (e.g. swap/offer shift) resolved in under 24 hours and strong weekly active use.

General Constraints

General Constraints identify any business or system constraints that will impact the manner in which the software is to be:

- specified
- designed
- implemented, or
- tested.

Time constraints

- The project must be completed within a two semester period and limited student resources.

Technology Constraints

- The system will be implemented as a web based application using widely supported technologies and a back end framework. It must be compatible for modern web browsers and responsive for both desktop and mobile

User and role Constraints

- The system must implement role base access control to ensure users only access features relevant to their roles.

Data and SecurityConstraints

- The system must handle user data securely. Sensitive information such as user information and log in credentials will be protected using encryption

2.3 Assumptions and Dependencies

List any assumptions that have been made during the initiation of the project. In addition, list any dependencies that may impact its success or the desired result.

Assumption

-The project team will have access to all necessary software and tools required for development.

-Users will have basic technical literacy to operate a web based platform.

-The defined user personas accurately represent the needs and challenges of the end-users.

-The team members will be available to their roles for the duration of the project timeline.

Dependencies

-(Internal)Frontend development for certain features is dependent on the completion of the corresponding backend APIs and database structures.

-(Internal)The finalization of the database schema is required before server side business logic can fully be implemented.

-(External)The project's success is dependent on the team members' continued collaboration and availability.

-(External)The project's direction, scope, and priorities may be influenced by external factors such as feedback from the primary instructor and insights derived from market data analysis.

3.0 Functional Requirements

This section describes specific features of the software project. If desired, some requirements may be specified in the use-case format and listed in the Use Cases Section.

3.1 <Functional Requirement or Feature #1>

3.1.1: Feature #1 - Set/Update Availability

Introduction	This feature allows users to set, view, and update their work availability.
Inputs	User provided availability data which shows start and end times, as well as selected days
Processing	The system validates the input, stores it in the database, and updates the user's availability records.
Outputs	Confirmation message and updated availability displayed on the user's profile.

3.1.2: Feature #2 - Schedule Management

Introduction	This feature enables managers to create, update, and assign schedules for users.
Inputs	The manager entered schedule details such as date and time that's assigned to the employee.
Processing	The system checks employee availability, assigns shifts, and updates the central schedule database.
Outputs	A page showcasing schedule for all users and notifications sent to affected employees.

3.1.3: Feature #3 - Viewing Availability and Schedules

Introduction	All users can view their personal availability and schedule information to stay informed about work times.
Inputs	Request from the user to view schedules or availability by selecting on schedule or availability tab button.
Processing	The system retrieves data from the database and formats it for display.

Outputs	Displayed schedule and availability information accessible to all users.

3.1.4: Feature #4 - Add Employee

Introduction	This feature allows managers to add new employees to the system.
Inputs	Employee details such as name, role, contact information when adding. Only the employee ID is required when removing.
Processing	When adding, the system validates and stores the new employee's information in the database. When removing, the system locates the employee record and deletes it while ensuring no schedule conflicts remain.
Outputs	A confirmation message showing that the employee has been successfully added. The updated employee list is reflected in the system.

3.1.5: Feature #5 - Remove Employee

Introduction	This feature allows managers to remove existing employees from the system.
Inputs	Only the employee ID is required when removing.
Processing	When removing, the system locates the employee record and deletes it while ensuring no schedule conflicts remain.
Outputs	A confirmation message showing that the employee has been successfully removed. The updated employee list is reflected in the system.

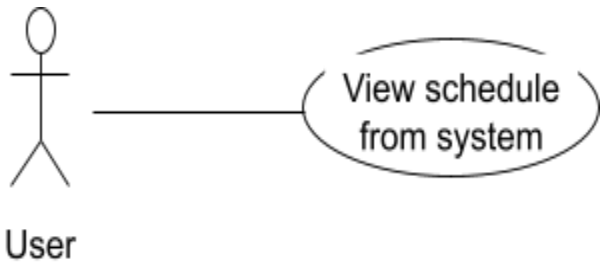
3.1.2:

3.2 Use Cases

3.2.1 Use Case #1:

Use case: View availability
This use case is to view the user's schedule on when they are working.

Diagram:



Brief Description
The user can view their schedule from the system.

Initial Step-By-Step Description
Before this use case can be initiated, the user must click on the schedule tab from the navigation bar.

1. Open up the website
2. Click on the schedule tab from the left side of the screen
3. Once clicked, the user will be presented with a calendar where they can view what days and set time they will be working.

3.2.2 Use Case #2: Update availability

Use case: Setting availability
This use case is to set or update the user's availability.

Diagram:

Brief Description
This use case purpose is for the managers to update an employee's availability. This will result in the manager to only schedule that specific employee depending on their availability.

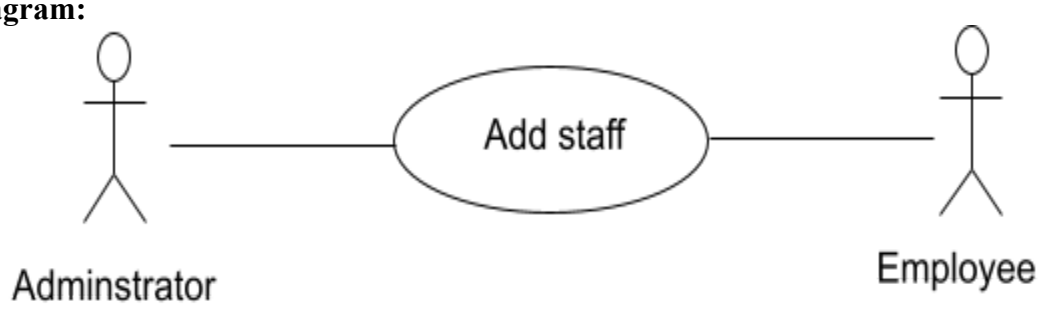
Initial Step-By-Step Description
Before this use case can be initiated, the manager must pick an employee from the employee's list which will lead them to the 'update availability' page where they can change the times and days of the week that they are allowed to work on.

1. Manager must click on the employee's tab to view employee's list
2. Click on the specific employee that the manager wants their availability to be changed.
3. Will be presented a page showcasing all days of the week and the manager can update the set time for all these days. They can also choose what days they aren't able to work on.

3.2.3 Use Case #3:

Use case: Add Employee
This use case is to add an employee

Diagram:



Brief Description
This use case purpose is for administrators to add an employee if hired, where they can register for an account.

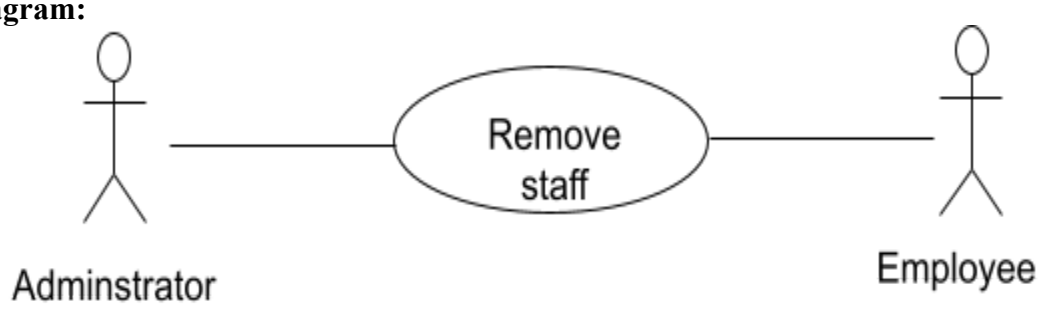
Initial Step-By-Step Description
Before this use case can be initiated, the administrator must go to the employee's tab and click on the 'Add Employee' button which will lead them to the registration page.

1. Administrator must click on the employee's tab to view employee's list
2. On the top of the list would show a button to add an employee
3. Once clicked on the button, it will lead them to the registration page where they can make a new account for that new employee.

3.2.4 Use Case #4:

Use case: Remove Employee
This use case is to remove an employee

Diagram:



Brief Description
This use case is for administrators to remove an employee if they are fired which deletes their account, preventing them from logging back into their account.

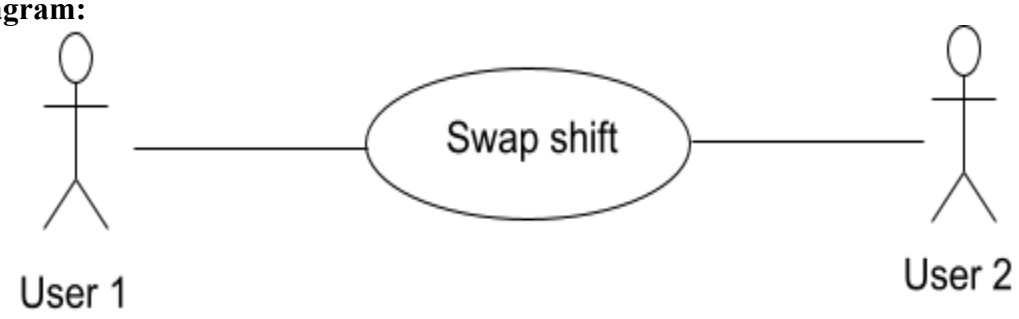
Initial Step-By-Step Description
Before this use case can be initiated, the administrator must go to the employee's tab. At the side of each employee will present a delete button to terminate their account.

1. Administrator must click on the employee's tab to view employee's list
2. At the side of the employee, the user should click on the Delete button
3. It will prompt them to confirm if they want to delete, they must click on 'Yes'
4. Once clicked, it will result in the account being terminated, preventing the user from logging back in.

3.2.5 Use Case #5:

Use case: Swap shift
This use case is to swap a shift.

Diagram:



Brief Description
This use case is for a user to swap a shift to another user that has the same role which then the user will have to wait until the manager accepts the shift swap between the two users.

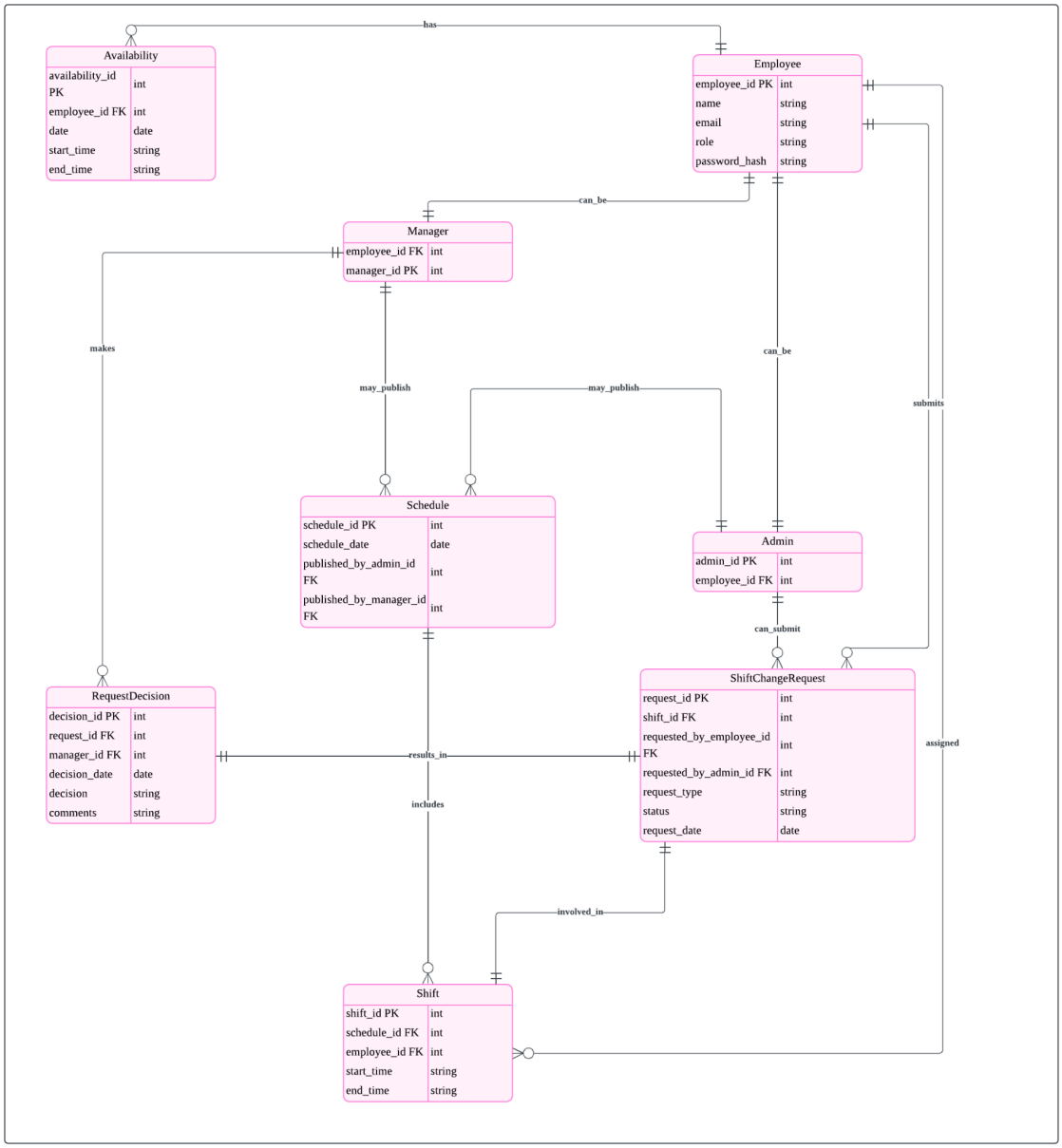
Initial Step-By-Step Description

Before this use case can be initiated, the user must go to their schedule and click on the specific day they are working on and swap that shift. It will then provide other users that they could possibly swap shifts with.

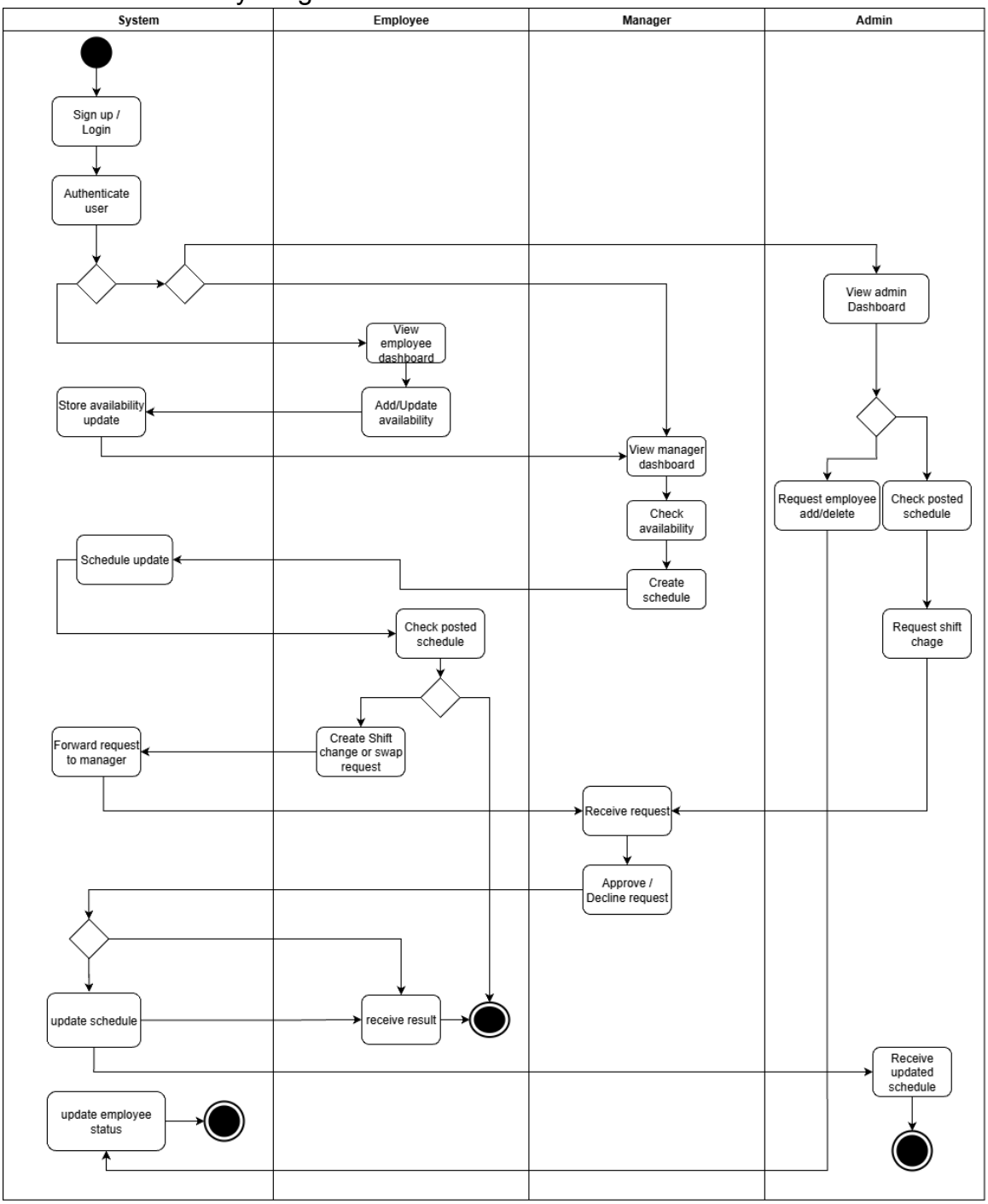
1. User must go on their schedule from the schedule tab
2. Click on the specific day they want to swap, there will be a button to swap shifts
3. Once user clicks on that button, it will show all the possible options of shifts from others that the user could swap with
4. They must click on a shift they want to swap, then both users will have to wait for the manager's approval before the shifts change in the system.

3.3 Data Modelling and Analysis

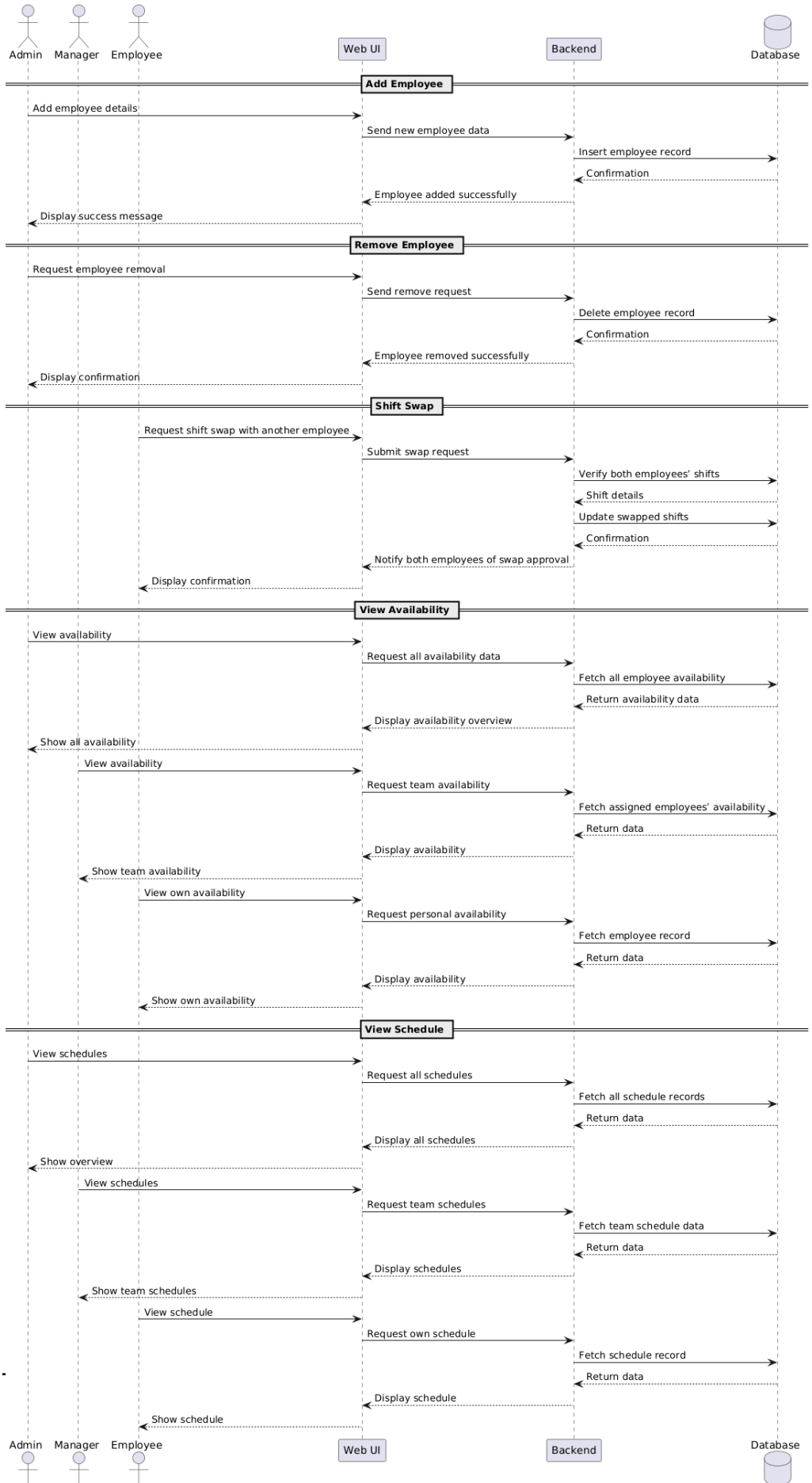
- Normalized Data Model Diagram



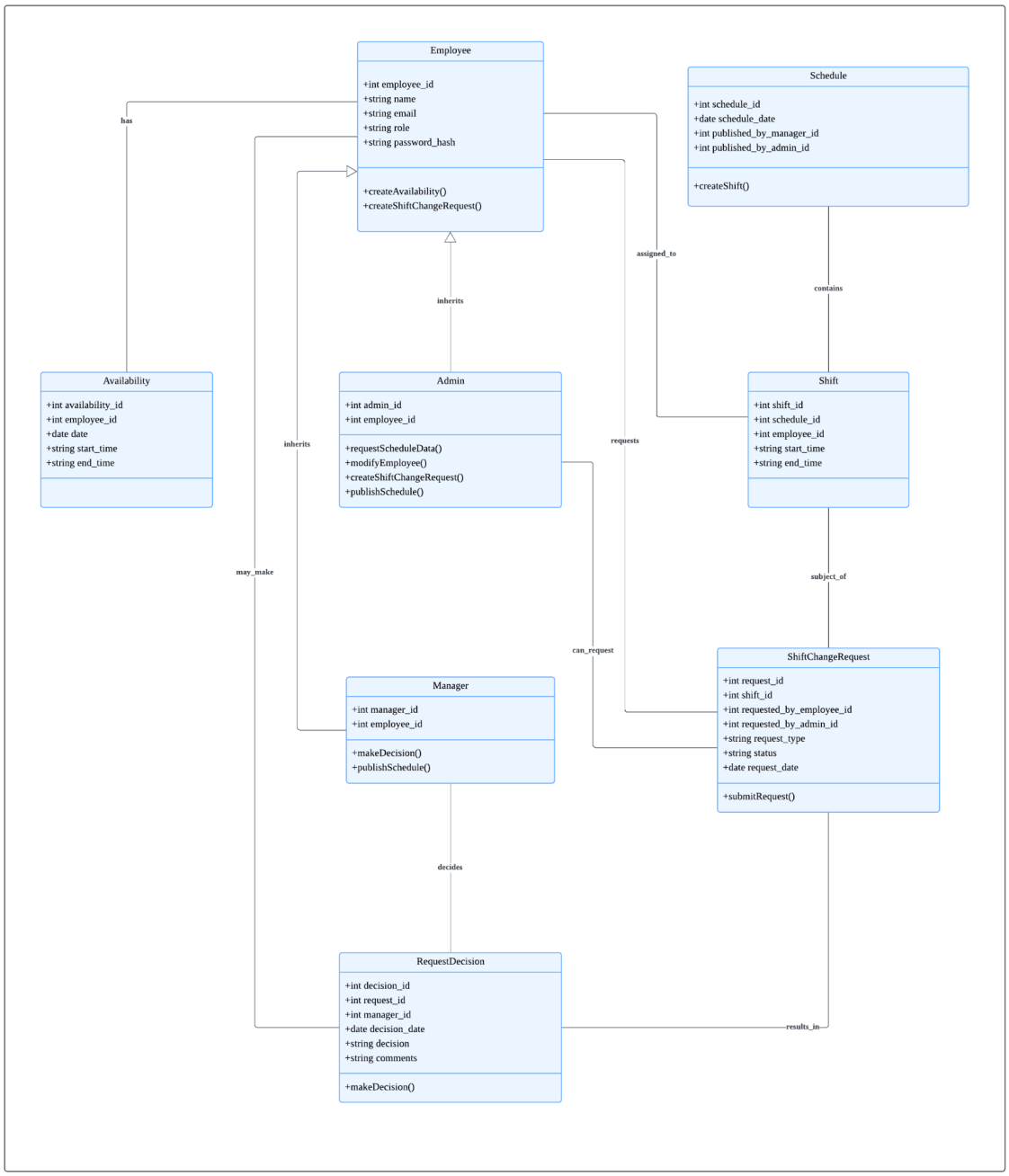
- Activity Diagrams



- Sequence Diagrams



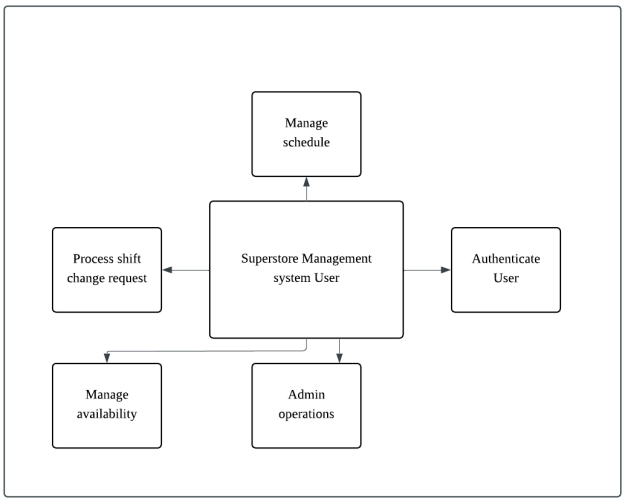
- UML Class Diagram



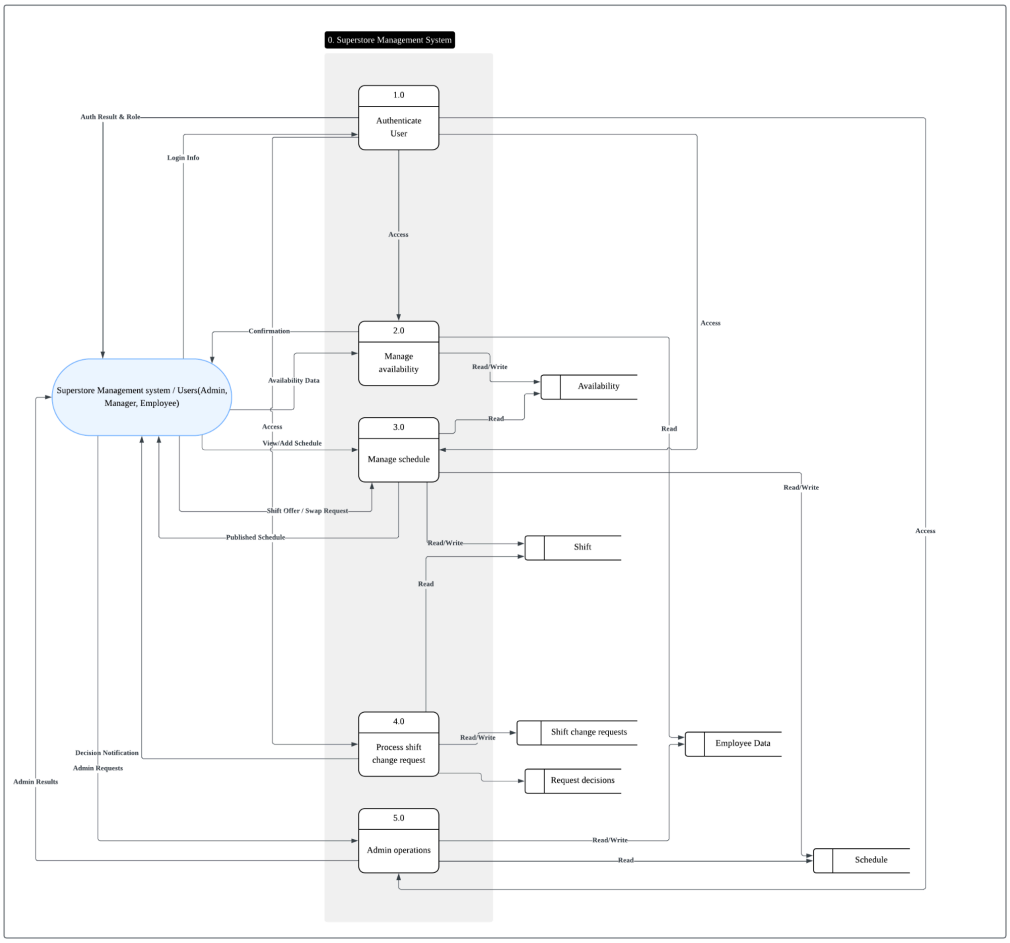
3.4 Process Modelling

- Data Flow Diagram

Level 0



Level 1



4.0 Non-Functional Requirements

The non-functional requirements for a system are typically constraints on the functional requirements – that is, not what the system does, but how it does it (e.g. how quickly, how efficiently, how easily from the user's perspective, etc.).

Non-functional requirements may exist for any of the following attributes – Performance, Reliability, Availability, Security, Maintainability, Portability.

Often these requirements must be achieved at a system-wide level rather than at a unit level. State the requirements in the following sections in measurable terms (e.g., 95% of transaction shall be processed in less than a second, system downtime may not exceed 1 minute per day, etc).

Security

- **Secure Login:** The system must provide a secure login mechanism.All user authentication transactions (login attempts) must use industry-standard encryption (**TLS 1.2 or higher**).
- **Authentication:** The system must use **role-based authentication** (employees vs. managers) to ensure users have correct and limited access to features.
- **Password Enforcement:**.The system must enforce strong password rules requiring a minimum length of **10 characters**, including a mix of uppercase, lowercase, numbers, and special characters.

Maintainability / Scalability

- The application must be built on a **scalable foundation** for future features. This is to allow for the later integration of features such as payroll, complex reporting, and multi-location management.
- The application code must adhere to a defined coding standard and contain developer documentation allowing a new developer to resolve a low-priority with initial investigation.

Reliability / Availability

- The system must maintain uptime and stability to ensure employees and managers have regular access to schedules and management functions.
- All approved shift schedules and employee availability data must be stored with 100% data integrity following any transaction, ensuring the schedule displayed to the employee matches the schedule published by the manager.
- The system must be available and operational for end-users (employees and managers) with an uptime target of 99.5% during core Superstore operating hours.

5.0 Logical Database Requirements

Will a database be used? If so, what logical requirements exist for data formats, storage capabilities, data retention, data integrity, etc?

1. Employees (id,name,email,roleids)
2. Roles(id,titles)
3. Locations(id,name,address,timezone)
4. Shifts (id,locationId, roleId, employeeId?, startAt,endAt,status)
5. Schedules(id,locationId, weekStart,publishedAt)
6. Requests (id,employeeid, type:offer/swap,targetShiftId,reason,status.managerid,decideAt)
7. Availability (id,employeeId, dayOfWeek,startAt,endAt,repeats)

And the database must support at least 10-20 users and 1-3 locations

6.0 Other Requirements

Additional requirements, if any.

7.0 Approval

The signatures below indicate their approval of the contents of this document.

Project Role	Name	Signature	Date
Lead Developer and Project ManagerProject Manager	Hans Casilao	HC	Nov. 5, 2025
System architect and QA lead	Soojeong Lim	SL	Nov. 5, 2025
Backend development and business analysisLead Developer	Hanson Bani Khaled	HBK	Nov. 5, 2025
Backend developer	Gurrmesh Singgh	GS	Nov. 5, 2025
Frontend developer and QA tester	John Patrick Buenaventura	JPB	Nov. 5, 2025